PROGRESS REPORT • MHD-WAAM THESIS • 2 – 9 SEPTEMBER 2026

The Arc Fired, and Nothing Melted

Phases 2 and 3, the three separate faults behind one null result, and a domain that is now verified

Baseline: roadmap rev. 4, 1 September 2026 — "Phases 0 and 1 complete, Phase 2 next, the arc has never fired"

THE WEEK IN SIX LINES

The arc source works. Goldak's double ellipsoid was implemented, verified arithmetically before any code was written, and delivers **99.97%** of its target 756 W steadily across 50,000 timesteps. Task A1 existed to establish exactly that, and it passes.

It produced no weld pool — 1 mm^3 of mushy metal against Zhao's ~8 mm bead, and not one cell anywhere above the liquidus. Chasing that null result is what the rest of the week was.

Three independent faults were behind it, and only one was physics. The domain was acting as an insulated crucible; an upstream solver defect meant no thermal boundary condition anywhere in the code was doing anything; and three of the measurement scripts were reporting numbers they had assumed rather than read.

The physics answer, once the other two were fixed: the arc alone saturates at about **832 K**, crossing the 815 K solidus by 17 K and falling 74 K short of the 906 K liquidus. GMAW melts the plate with superheated filler droplets, not with the arc, and the wire inlet is therefore on the critical path rather than in Phase 4.

The domain is now verified independent of its far boundary and is larger than the source paper's in all three directions. That claim is backed by a measured 0.2 K rise at the far face and a bit-identical control run, not by an argument.

The recurring failure mode has a name now. Seven separate parameters this week appeared to be configured, silently were not, and produced confident wrong answers — the last of them, an absent magnetic switch defaulting to on, cost 1.79x on every run for a week. Every tool written since refuses rather than assumes.

1. Where the plan stood on 1 September

Roadmap rev. 4 closed Phases 0 and 1. The toolchain was settled on OpenFOAM ESI v2506 with the `OpenFoam_com_main` branch of laserbeamFoam at commit `7578023c`, chosen over DEDFoamv2 because it carries the newest MTHD magnetic module and because DEDFoamv2's only unique feature is a Lagrangian powder cloud, which is irrelevant to a process that deposits wire.

Two validation gates had passed. **Gate A** reproduces the analytical Hartmann profile in a non-chaotic, one-dimensional MHD case; it is *decisive*, meaning a difference in it can be attributed to a cause. **Gate B** runs Plate2D with the magnetic module, VOF, melting, latent heat, Darcy damping and Marangoni all active at once; it is *sensitive* rather than decisive, because melt-pool flow is chaotic and it amplifies any real change into a visible one. Neither gate is sufficient alone, which is why both are kept.

The physics ledger at that point counted **ten capabilities that ship working** (VOF, enthalpy-porosity melting, latent heat, Darcy damping, CSF surface tension, Marangoni, Boussinesq buoyancy, Lorentz force, Joule heating, and interface blending of the electromagnetic properties), **three changes needing dictionaries only**, and **four needing solver patches**. Two of the four were complete: Edit 1, a two-branch temperature-dependent thermal conductivity, and Edit 2, the Goldak arc source. The status note ended plainly: *"The arc has never fired."*

THE CLAIM THAT DID NOT SURVIVE THE WEEK

Rev. 4 said six of the seven changes would be `codedFvOption` or `codedFixedValue` snippets needing no solver surgery. Reading the source had already moved three of those into code, because `TEqn` accepts no `fvOptions` and surface forces need the balanced-force treatment. This week added a fourth, unplanned solver change — Edit 8 — and identified two more. **The seven-change ledger is now ten.**

This is worth stating because it is the kind of estimate that decides whether the 20-week plan fits the runway. Solver patches cost more than dictionary entries, and the ratio has moved against us.

2. Phase 2 — the property set

Background. Every quantity in this model is downstream of the material properties, and three in particular. Thermal diffusivity $a = k/(\rho \cdot c_p)$ sets how far heat travels in a given time, and therefore how large the domain has to be. Electrical conductivity σ_e sets both the Lorentz force and the Joule heating, so it scales the entire magnetic contribution that this thesis exists to measure. Surface tension and its temperature derivative set the Marangoni flow, which for aluminium is the dominant convection mechanism in the pool.

Zhao's Table 4 gives c_p, k and viscosity as "temperature dependent" with a citation rather than as numbers, so those had to be chased back to the source rather than guessed. The values now in `constant/transportProperties`:

PROPERTY	VALUE	ROLE
Density rho	2650 kg/m^3	Mass, buoyancy reference
Specific heat c_p	940 J/kg/K	Calibrated by closing the energy balance on the first run, not taken from a table — see the note below
Conductivity k	two-branch, solid and liquid (Edit 1)	Diffusivity; the branch used for domain sizing gives a = 6.22e-5 m^2/s
Solidus / liquidus	815 K / 906 K	Mushy range; ExaCA needs both
Latent heat L_f	3.58e5 J/kg	Enthalpy-porosity source
Surface tension	0.85 N/m	CSF force at the free surface
dsigma/dT	–1.55e-4 N/m/K	**Negative**, so the surface flows outward from the hot centre and the pool comes out wide and shallow rather than deep
Arc efficiency eta	0.7	135 A x 16 V x 0.7 = 1512 W
Gas conductivity ratio	3.65	Sets the interface blending floor

c_p = 940 J/kg/K deserves a word, because it was not read from a handbook. It was obtained by requiring that the thermal energy in the metal equal the energy the arc reports delivering — that is, by forcing the energy balance to close on a run where nothing could escape. That makes it a fitted constant rather than a

measured one, and it is flagged as such. The residual it leaves is visible in every run as the `E kept` column settling at about 100.5% rather than exactly 100%.

SILENT DEFAULT NUMBER ONE: sigma_e_gas_ratio

The gas electrical conductivity was supposed to be set by a ratio parameter. It was being read from a key that the solver never looked at, so the value in the dictionary had no effect whatsoever and the built-in default was used in every case. Nothing failed; the runs completed and produced plausible fields.

This is the first instance of what turned out to be the week's dominant failure mode, and it is worth naming precisely: **a parameter that appears configured, silently is not, and produces a confident wrong answer.** There is no error, no warning, and no way to notice from the output.

3. Phase 3 — the Goldak arc source

Background: why a double ellipsoid. A welding arc does not deposit its heat on a point or uniformly over a disc. It deposits it into a volume under the torch, and that volume is not symmetric front-to-back: the leading edge of the arc meets cold metal while the trailing edge meets metal already heated by the pass, so the heat distribution is compressed ahead of the torch and stretched behind it. Goldak's model captures this with two half-ellipsoids joined at the torch plane, each a Gaussian in three dimensions, with different semi-axes fore and aft. It is the standard welding heat source and it is what Zhao uses, as Eq. 27.

The parameters, from Zhao Table 4 and obtained there by matching simulated and measured pool cross-sections:

a_f = 3 mm forward semi-axis, along travel
a_r = 6 mm rearward semi-axis, twice the forward
b = 4 mm transverse semi-axis
c = 4 mm depth semi-axis
f_f = 0.667, f_r = 1.333 fore and aft weights, $f_f + f_r = 2$ by construction

torch centre $(x_0 + V_T t, y_0, z_0)$, V_T = 0.02 m/s = 120 cm/min

Two things were verified before a line of solver code was written. First, the ellipsoid was integrated numerically over the domain to confirm it sums to the stated power. That is how the factor-of-two half-ellipsoid convention was caught: Goldak's normalisation assumes a full ellipsoid in depth, but only the half below the surface lies in metal, so a naive implementation delivers half the intended power and does so silently. Second, the source renormalises: it measures how much of the ellipsoid actually fell inside metal and scales up to hit the target exactly, then **prints both the raw and the renormalised figure**. A misconfiguration announces itself in the log instead of being inferred later from a wrong pool.

WHAT THE ARC SOURCE DELIVERS

99.97% of its 756 W target before renormalisation, constant across 50,000 timesteps, with mass conserved to ten digits. The 0.03% shortfall is the Gaussian tail crossing the domain edge, which is the correct behaviour and not an error.

756 W and not 1512 W because the domain is a symmetric half. The arc source is told which it is and halves accordingly; getting this wrong in either direction would be a factor-of-two error in the only energy input the model has.

4. The first arc run, and the null result

With the arc firing, task A1 built one concrete case by hand — `waam-arc-01`, a bare-plate autogenous bead-on-plate pass with no filler, the laser at zero power and the magnetic module active but with B = 0. Deliberately no template tokens: debugging the physics setup and the token substitution simultaneously is how you end up unable to say which one is broken.

It ran to completion and produced **a pool of 1 mm^3**. Zhao's bead is about 8 mm wide. That is not a small discrepancy to be tuned away; it is a qualitatively different outcome, and there were three candidate explanations that had to be separated rather than guessed between.

CANDIDATE	HOW IT WAS TESTED	VERDICT
The domain is too small and is heating up as a whole, so nothing localises	Compute the diffusion length; compare predicted and measured bulk heating rate; rebuild larger and graded	**TRUE** — and it was distorting the answer in the flattering direction
A boundary condition is wrong, so heat is escaping or being trapped	Set the walls to 2000 K and check whether anything responds	**TRUE, and far worse than suspected** — no thermal boundary condition in the solver was applied at all
The arc genuinely cannot melt this plate, and the physics is missing an energy source	Close the energy budget by hand: arc power against the enthalpy needed to melt the observed volume, then against the droplet enthalpy the model omits	**TRUE** — and it is the finding, not the fault

All three were real and they were compounding. The discipline that made this tractable is the same one used for the gates: **change one thing, and make the tool refuse to run if the thing you meant to hold fixed has moved.**

5. Fault A — the domain was a crucible

Background: how far does heat go. Conduction is diffusive, so the distance a thermal disturbance travels in time *t* is not proportional to *t* but to its square root:

$$L = \sqrt{4 \cdot a \cdot t}, \quad a = k/(\rho \cdot c_p) = 155/(2650 \times 940) = 6.22\text{e-}5 \ m^2/s$$

$$t = 0.5 \ s \rightarrow L = 11.2 \ mm$$
$$t = 2.0 \ s \rightarrow L = 22.3 \ mm$$

A domain boundary closer than about two diffusion lengths is inside the thermal field, and whatever condition is imposed there becomes part of the answer. The original domain was 25 x 8 x 9 mm — its far side sat at 8 mm, well inside the 11.2 mm the heat reaches in half a second. The consequence was measurable: **the bulk temperature of the whole plate rose at 252 K/s**, which is a crucible warming up, not a workpiece conducting heat away from a weld.

Why that flatters the result rather than spoiling it obviously. The physically meaningful quantity is peak minus bulk — the local excess the arc can sustain above whatever the plate happens to sit at. When the surroundings warm up alongside the hot spot, that difference is compressed, and the arc looks weaker than it is. The first analysis reported a saturated excess of 431 K, and the correct figure is 532 K. The error was in the conservative direction for the conclusion but it was still an error, and it would have gone into the thesis.

The rebuilt domain

The fix is not simply a bigger uniform mesh. 60 x 25 x 9 mm at 0.5 mm cells is 120 x 50 x 18 = **108,000 cells**, exactly twice the graded count, with the extra spent resolving far-field conduction that has no structure in it. Instead the mesh is **graded**: fine where the physics is and expanding toward the boundaries. Note that grading saves a factor of two here and no more, because the time step is set by the Courant number on the *smallest* cell and that cell is 0.5 mm either way. Grading buys memory and per-step cost, never a longer step.

DIRECTION	EXTENT	TREATMENT
x, along travel	–25 to +35 mm (60 mm)	Multi-section grading: coarse, then 0.5 mm through the region the torch crosses, then coarse. Both ends are coarse, which is why blockMesh's "first and last cell" report shows them as equal and looks uniform
y, transverse	0 to 25 mm (half domain)	0.5 mm at the symmetry plane, expanding by 4 to the far boundary. The symmetry plane at y = 0 halves the cost and is valid because B = 0
z, depth	–6 to +3 mm	**Ungraded**, 0.5 mm throughout. 6 mm of substrate, 3 mm of argon above it

That gives **88 x 34 x 18 = 53,856 cells**, cell volumes from 0.12 to 1.91 mm^3. The ratio is **16**, not the 64 a factor-of-4 expansion would suggest, precisely because z is not graded — the ratio is the linear expansion squared, not cubed. That mattered: an earlier claim of "about 60x" was wrong and had been written into a measurement script's documentation.

THE CRUCIBLE IS GONE, AND THE PREDICTION WAS EXACT

Predicted bulk rise for the new domain over 0.5 s: **+17 K**. Measured: 300.0 K to **317.0 K**. The heating rate fell from 252 K/s to **34 K/s**.

An exact agreement between a hand calculation and a simulation is worth more than it looks. It says the energy accounting, the mesh volumes and the material properties are all mutually consistent — three things that were separately suspect — because an error in any one of them would show up here.

6. Fault B — every thermal boundary condition was inert

Background: how OpenFOAM stores a field. A `volScalarField` has two parts. The *internal field* holds one value per cell. The *boundary field* holds one value per face on each patch, and each patch carries a type that says how its face values are obtained. `fixedValue` holds them at a number. `zeroGradient` copies the adjacent cell's value to the face, so the normal derivative is zero. `calculated` means "something else will write these" — **it has no evaluation rule of its own.**

That last point is the whole defect. In `createFields.H` the conductivity and specific heat are constructed like this:

```
volScalarField kappa
(
    IOobject(...),
    mesh,
    dimensionedScalar("kappa", dimensionSet(1,1,-3,-1,0), 0.0)
);
```

The dimensionedScalar constructor gives every patch a 'calculated' boundary field initialised to 0.0. Nothing later changes that.

`updateKappaCp.H` fills the interior each timestep and then calls `kappa.correctBoundaryConditions()`, which is the correct thing to call — but `calculated` has no evaluation rule, so that call does nothing at all and the boundary values remain zero for the entire run.

Why zero coefficients kill both transport routes

The temperature equation moves heat in two ways, and a zero coefficient at the boundary disables each of them there:

```
– fvm::laplacian(kappa, T) conduction. The face flux is
                           kappa_f · A · dT/dn. With kappa_f = 0 there is no flux
                           whatever dT/dn the dictionary asks for

+ fvm::div(rhophicp, T) advection. rhophicp is built from
                           fvc::interpolate(cp), which is also 0 at
                           the boundary, so no enthalpy crosses either
```

The consequence is that **the domain was a perfectly insulated box whatever the dictionaries said**, in every case, in every solver in the family. This was confirmed directly: walls held at a fixed 300 K, walls made adiabatic, and walls held at 2000 K all produced output identical to nine significant figures, and the energy balance closed at 100% because nothing could ever leave.

> **HOW IT WAS FOUND, AND WHY THE REASONING MATTERED MORE THAN THE BUG**
>
> The trail started from an inconsistency, not from reading the source. Switching the walls from `fixedValue 300` to `zeroGradient` changed the answer by less than one part in 10^8, and the written output confirmed the new condition had been applied. But the bottom cell layer was at 524.8 K with a 300 K clamp 0.25 mm away, which implies a flux far larger than the arc input. Both facts could not be true.
>
> The test that settled it was deliberately absurd: **set the wall to 2000 K.** If the bottom layer does not respond to that, no boundary condition is being applied. It did not respond.
>
> **The fix** is to pass `zeroGradientFvPatchScalarField::typeName` as the patch-field type to both constructors, so `correctBoundaryConditions()` has something to do. That is also the physically correct choice: the conductivity and specific heat at a wall are those of the material touching it. **Verified:** the same 2000 K test now moves the bottom layer from 313.1 K to **1969.8 K**.
>
> **Every wall-thermal-BC result taken before this is void**, and the wall-BC sensitivity study planned for Phase 9 would have reported "no sensitivity" — a false null indistinguishable from a real one.

This is **Edit 8**, and it is the first change in this project found by testing rather than planned. Unlike Edits 1 and 2 it is deliberately **not** bit-identical: there is no legacy path to preserve, because the entire point is that heat can now cross a boundary where before it could not. Both gates were therefore re-baselined rather than expected to match. Gate A, being isothermal, stayed bit-identical at Ha = 15.810 and 1.3869e-02. Gate B moved: melt volume by +0.21%, inside its 0.5% noise floor, and mean velocity by +9.1%, which is above its 2.6% floor and is therefore a real change. Those are the new baseline.

7. Fault C — the instruments were lying

Three of the scripts used to judge the model were reporting numbers they had assumed rather than measured. None of them failed; all three produced plausible output.

7.1 The pool measurement assumed a uniform mesh

`pool_stats_waam.py` derived each cell's position from its index and treated every cell as the same size. That is true of a uniform block and false the moment the mesh is graded, where volumes here span a factor of 16. Pool volumes and depths computed that way would have been confidently wrong on the very mesh built to fix fault A.

It now reads OpenFOAM's own cell centres and volumes — written by `postProcess -func writeCellCentres` and `writeCellVolumes` — and **exits rather than falling back on an assumption** if they are absent. It also cross-checks the geometry against the physics before reporting anything: at the first time, every cell below z = 0 must be metal and every cell above must not be. That catches a mismatched mesh before a single number is printed.

7.2 The velocity column was reporting the shielding gas

The reported maximum velocity was taken over every cell in the domain, which means over both phases. Buoyant argon above the hot spot moves an order of magnitude faster than anything in the metal, and it does so from the first timestep — long before anything melts. The first analysis recorded **"max|U| reached 0.256 m/s, a plausible Marangoni velocity"** at a time when **zero cells were molten**. It was reading a velocity from a phase that did not exist yet. Velocity is now reported per phase, and the control run confirms the diagnosis: at 0.02 s the metal is at **0.0000 m/s** and the gas at 0.0067 m/s.

7.3 Gate A was being scored against the wrong Hartmann number

The Hartmann number Ha = B·L·sqrt(sigma/mu) is the ratio of magnetic to viscous forces, and it selects which member of a family of velocity profiles the analytical solution predicts — a plug profile at high Ha, a parabola at Ha = 0. The checking script defaulted to sigma = 1 and mu = 1. The tutorial case sets an electrical conductivity of 1000, hidden behind an `#include` that the script never read.

INVOCATION	Ha	MAX DEVIATION	VERDICT
Defaults, sigma = mu = 1	0.500	6.2903e-01	**FAIL**
Correct, sigma = 1000	**15.810**	**1.3869e-02**	**PASS**

A genuine plug profile was being compared against a parabola and scored as a **63% failure on a case that passes at 1.4%**. Worse, the correct invocation was recorded nowhere in the repository: Gate A was reproducible only by remembering a command line, and a FAIL baseline was committed and pushed before this was caught. The script now reads the properties from the case, prints the provenance of every number it used, and **refuses to run** if a value can be neither read nor supplied — because assuming 1 is exactly what turned a pass into a fail.

8. The physics result, corrected

With the domain enlarged and the boundary conditions actually applied, the arc-only run gives a clean and interpretable answer. The excess above bulk converges geometrically, which is the signature of a diffusion problem settling to steady state:

TIME	PEAK T	MEAN T	PEAK – MEAN	INCREMENT
0.1 s	702.0 K	303.4 K	398.6 K	—
0.2 s	773.4 K	306.8 K	466.6 K	+68.0 K
0.3 s	806.3 K	310.2 K	496.1 K	+29.5 K
0.4 s	826.4 K	313.6 K	512.8 K	+16.7 K
0.5 s	838.9 K	317.0 K	**521.9 K**	+9.1 K

Each increment is about 1.85 times smaller than the one before. Summing the remaining geometric tail, 9.1/(1.85 − 1) = 11 K, gives a saturated excess of **about 532 K** and, on a substrate cold enough to stay near 300 K, a peak of **about 832 K**.

THE ARC REACHES THE SOLIDUS AND STOPS THERE

832 K is **17 K past the 815 K solidus and 74 K short of the 906 K liquidus**. The run agrees with the extrapolation: at 0.5 s exactly **8 cells** are above the solidus, 1.00 mm^3 in total, 0.75 mm deep, first appearing at t = 0.36 s — and **not one cell anywhere in the domain is above the liquidus**. A mushy skin, not a pool.

This supersedes the first analysis, which reported the arc falling 84 K *short* of the solidus. The conclusion survives and the number does not: the arc grazes the solidus rather than missing it, but nothing that can be called a weld pool forms and no thermal history worth giving to ExaCA comes out of it.

Where the missing energy is: the droplets

GMAW does not melt the workpiece with the arc. It melts it with superheated filler droplets, which Zhao's assumption 1 models as hot liquid entering through a virtual tube at 1506 K. Working out how much energy that is takes only the feed rate and the enthalpy needed to bring solid wire at ambient to liquid at the wire-tip temperature:

```
wire 1.2 mm dia at 0.15 m/s -> 0.450 g/s (= 900 cm/min)

enthalpy above 300 K, per kg
  cp(815–300) 940 x 515 = 484,100 heat solid to the solidus
  Lf = 358,000 melt it
  cp(906–815) 940 x 91 = 85,540 cross the mushy range
  cp(1506–906) 940 x 600 = 564,000 superheat to the wire tip
                                ---------
                                1.492 MJ/kg

droplet power = 4.495e-4 x 1.4916e6 = 671 W (335 W in the half domain)
arc power = 1512 W (756 W in the half domain)
droplets add 44.4% ON TOP
```

And they do not merely add power — they arrive **already molten**. That energy needs no conduction to become pool; it *is* pool. Scaling the saturated excess by the added power puts the peak at about 1066 K, well past the liquidus. That scaling errs in both directions at once — low, because droplets deposit into a few cells while the arc spreads over the whole ellipsoid; high, because latent heat is absorbed isothermally exactly where the estimate assumes the temperature keeps rising — but the sign is not in doubt.

9. Boundary conditions: what the paper actually does

Once Edit 8 made boundary conditions live, choosing them stopped being cosmetic. Two questions had to be separated, and they had been sharing one patch.

9.1 Splitting a modelling choice from a truncation

The original case had a single patch called `walls` covering both the underside of the plate and the far side of the domain. Those are different in kind. The **underside is real**: it is where the plate meets air or a fixture, heat genuinely reaches it, and what condition to impose is a physical modelling question. The **far side is imaginary**: nothing is there, the plate continues past it, and if the domain is large enough the condition should not matter at all.

Sharing one patch made the domain-independence test impossible — changing it changed both a physical choice and a numerical truncation simultaneously, and the two effects arrived in one number with no way to separate them. They are now `substrateBottom` and `farField`, and the split is exact: 2992 + 1584 faces against the old patch's 4576.

The data justifies the split immediately. At 0.5 s the far face is at **300.2 K** — 0.04% of the peak rise, so no condition there can matter — while the plate underside is at **517.2 K**, where the condition matters a great deal. Merged, flipping the patch would have produced a large change driven entirely by the bottom, and the natural conclusion would have been "the domain is too small", which is false.

9.2 Zhao's wall condition is neither of the obvious two

Table 3 of the paper gives the wall boundaries as Eq. 29 — convection *and* radiation to ambient, with a heat transfer coefficient h = 80 W/m^2/K. That is a Robin condition, and it is neither adiabatic nor fixed-temperature. Comparing the three at the plate underside, now at 517 K:

CONDITION	FLUX REMOVED AT 517 K	ERROR AGAINST ZHAO
Zhao, h = 80 plus radiation	~17.4 kW/m^2 (+ ~0.9 radiative)	—
`zeroGradient`, what we use	0	–17 kW/m^2
`fixedValue 300`	k·dT/(dz/2) = 155 x 217 / 0.25 mm = **134 MW/m^2**	**+134,000 kW/m^2, about 7,700x too much**

Adiabatic is the better approximation by four orders of magnitude. `fixedValue 300` is not a conservative bound — pinning a face a quarter of a millimetre from a 517 K cell manufactures a gradient of 870,000 K/m that nothing in the experiment produces. The size of the error we accept by staying adiabatic, taking the peak temperature across the entire bottom face as a ceiling, is 80 x 217 x 1.5e-3 m^2 = **26 W against 756 W, 3.4%**, and realistically about 1%. The Robin condition is recorded as **Edit 10, deferred**: it is not needed at 0.5 s and becomes necessary once runs are long enough for the underside to matter.

9.3 The domain against the paper's

	ZHAO'S METAL SUBDOMAIN	OURS
length, x	50 mm	**60 mm**
half-width, y	15 mm	**25 mm**
depth, z	5 mm	**6 mm**
wall condition	convective + radiative, h = 80	adiabatic (Edit 10 deferred)
run length	2 s	0.5 s

Larger in every direction. That also explains why Zhao needs the convective condition and we do not: at 2 s the diffusion length is 22.3 mm and their 15 mm boundary is well inside the thermal field, so it has to shed heat realistically. At 0.5 s ours is not.

10. Verification: the domain is independent of its far boundary

The `far` column reads 300.0 K until 0.40 s and 300.2 K at 0.5 s. That settles the question directly rather than by argument: while the near-wall cell is at ambient there is no gradient for any condition to act on, so `zeroGradient` (zero flux) and `fixedValue 300` (flux proportional to T – 300 = 0) pass the same zero flux and are mathematically identical there.

A control run confirmed the patch split itself changed nothing. Same mesh, same decomposition, both patches adiabatic, 0.02 s:

	REFERENCE (walls)	SPLIT (substrateBottom + farField)
max T	497.7 K	**497.7 K**
mean T	300.7 K	**300.7 K**
bottom	303.4 K	**303.4 K**
far	300.0 K	**300.0 K**
cells above solidus	0	**0**
E kept	102.2%	**102.2%**

Every printed digit identical. This is not a trivial check: melt-pool flow is chaotic, a round-off-level difference grows into a visible one within milliseconds, and splitting a patch reorders the summation of boundary faces. Bit-level agreement at 0.02 s says the reordering had no effect while the problem is still nearly linear conduction, which is the strongest statement available.

11. Repository and two-person working

The work was spread across a public fork of the upstream solver and a separate private repository, which meant the code on the machine and the code in version control had different shapes. It is now **one private repository** matching the working directory exactly, with the solver source vendored into `src/lbf-com` rather than carried as a submodule.

Vendoring loses the upstream commit history, so that history is replaced by an explicit record: `src/lbf-com-PROVENANCE.txt` names the base commit `7578023c` and the two commits carried on top of it. The public fork is kept as a frozen archive rather than deleted, because it is the only place the original branch history exists.

The two-track division

With a thesis partner joining, the work is split along the one seam that genuinely exists in this model. **Track A** is the melt pool — heat source, materials, mesh, wire, free surface, everything that decides whether a bead forms. **Track B** is electromagnetics — current injection, applied field, Lorentz force, the Hartmann gate. They meet only at the momentum equation and at the case matrix.

Four rules keep the two sets of results comparable, and they exist because of the chaotic sensitivity established in Phase 0: identical solver commit, identical decomposition, identical schemes, and integral quantities only — never cell-by-cell comparison. A file-ownership map prevents the two tracks editing the same source.

12. The pattern: seven silent defaults in one week

Every fault above shares a structure. A parameter appears configured, silently is not, and the run completes and produces a plausible answer. There is no error and no warning. Listing them together makes the frequency visible in a way that meeting them one at a time does not:

#	WHERE	WHAT IT DID
1	`sigma_e_gas_ratio`, Phase 2	Read from a key the solver never looked at; the dictionary value had no effect
2	`kappa` and `cp` boundaries, Edit 8	Every thermal boundary condition in every case inert; a 2000 K wall indistinguishable from perfect insulation
3	`hartmann_check.py`	Assumed sigma = mu = 1; scored a passing benchmark as a 63% FAIL, and that FAIL was committed
4	`pool_stats`, uniform-mesh assumption	Would have reported wrong pool volumes and depths on the graded mesh built to fix a different fault
5	`max|U|` over both phases	Reported buoyant argon as "a plausible Marangoni velocity" while zero cells were molten
6	`setFields` with no `0/` directory	Warned "Field alpha.metal not found" and did nothing; the whole domain would have stayed argon and the run would have completed
7	`electromagnetics`**, absent from transportProperties**	**The most expensive one.** The key was not in the dictionary at all, so the solver used its internal default, which is *true*. Every arc run since the source was built solved an induction equation and a magnetic divergence-cleaning step at every timestep for a field that is identically zero: **15 of 19 linear solves, and two thirds of the elliptic ones**. Found by asking the log what it had solved, not by reading a dictionary. Cost **1.79x** on every run for a week. No result is invalid — B = 0 makes the Lorentz force and Joule heating identically zero — but a 117-minute run should have been 67

THE RULE ADOPTED

When a parameter is supposed to matter, set it to an absurd value once and confirm the model reacts. That single test found Edit 8, after weeks in which every case silently ignored its thermal boundary conditions.

And a tool that cannot determine a value must refuse, not assume. Every script written or rewritten this week now exits with an explanation rather than falling back on a default: `pool_stats_waam.py` will not guess a mesh, `hartmann_check.py` will not guess a conductivity, `setbc.py` will not run a one-variable test whose control has moved, and `make_waam_case.py` aborts if the donor `controlDict` is missing any of the four time-step controls.

The cost of the alternative is now measured: a 63% FAIL baseline reached the repository, and a physics conclusion with a wrong number reached a project document.

13. Where the model stands

CHANGE	STATUS
Edit 1 — two-branch thermal conductivity	**Done, verified, committed.** Solver prints `Metal kappa: two-branch` at startup
Edit 2 — Goldak arc heat source	**Done, verified, committed.** 99.97% of target power, constant over 50,000 steps
Edit 3 — arc pressure	Not started. Interface Courant number is identically zero, which is correct while this does not exist
Edit 4 — wire inlet	**Designed this week, on the critical path.** Mass-deposition form chosen; resolved droplet deferred to the refined mesh
Edit 8 — kappa and cp boundary fields	**Done, verified, committed.** Found by testing, not planned. Both gates re-baselined
Edit 9 — ExaCA thermal-history writer	**Identified, unplanned.** At G ~ 1e5 K/m the mushy zone is crossed in 45 ms, giving only ~2 samples at the current write interval. laserbeamFoam has no such writer
Edit 10 — Robin wall condition, h = 80	**Identified, deferred.** Needed once runs are long enough for the plate underside to matter
Current injection	Not started. Phase 6, and the novel contribution

Verified: the arc delivers its power; the mesh and the energy accounting are mutually consistent to the kelvin; the domain is independent of its far boundary over 0.5 s and is larger than the paper's; both regression gates pass on the current solver; the patch split changed nothing. **Assumed and flagged:** c_p is fitted rather than measured; walls are adiabatic rather than convective. **The arc power figure has since been re-measured on the 71,808-cell mesh at 99.968%, renormalised x1.00032 to exactly 756 W** — it carries over from the old mesh unchanged, and a renormalisation that small means the ellipsoid is essentially fully contained in metal.

14. Corrections to the plan

- **Run duration is an output, not an input.** The scope narrowed this week from 2 s to "long enough to form a bead whose thermal gradient is usable". The pool first appears at 0.36 s with the arc alone; with droplets it will appear far sooner, and the right end time is whatever gives a quasi-steady bead, determined by running rather than by decree.

- **Edit 4 moves from Phase 4 to the critical path.** It was scheduled as a dictionary-only convenience. Nothing downstream can be validated without it, because no weld pool exists until the wire does.
- **The A1 acceptance criterion was mis-specified.** "A pool forms" was written into the gate for a model that physically cannot form one. The correct A1 gate is the power-delivery check, which passed.
- **ExaCA needs a solver-side writer, and it is a Phase 8 prerequisite.** This is new work that no phase budgeted for.
- **A cost anchor now exists.** Zhao ran 256,512 cells at 0.25 mm minimum with dt = 4e-5 on 16 cores for about 100 hours to simulate 2 s of a 40 mm bead. Refining our mesh from 0.5 mm to 0.25 mm multiplies cells by 8 and halves the time step: **roughly 16x the current cost**. That is the number for the HPC application.
- **The solver-surgery estimate has moved against us**, as described in section 1. Seven changes are now ten, and four of them touch source.
- **The magnetic module costs 1.79x, not 2.7x — and Phase 7 is cheaper than budgeted.** Rev. 4 says to multiply every compute estimate by 2.7 when the field is on. That figure was measured on *Plate2D*, which is two-dimensional, where the pressure Poisson equation is small and cheap so the magnetic solves dominate. Measured on this 3D case, with a 1650:1 density jump across the metal/argon interface making `p_rgh` the genuinely hard equation, the same module costs **1.79x** (0.1398 against 0.0806 s per timestep). The 13-case magnetic matrix is therefore about a third cheaper than planned. Re-measure on a real WAAM case before that number goes into the HPC application, but budget against 1.79 rather than 2.7.

15. Next: Edit 4a, the deposition source

The decision taken is to build the simple mass-deposition form now and defer the resolved droplet to the refined production mesh. The reason is resolution: Zhao's globular transfer detaches roughly every 28 ms and needs about 0.25 mm cells to resolve. At the current 0.5 mm the wire cross-section spans only about 2.3 inlet faces in the half domain, and that count jitters between 2 and 4 as the torch crosses the grid.

That jitter forces the two central design decisions. First, the source **normalises to the mass flow** rather than setting a velocity — the same renormalisation the arc already applies to power — so the face count can jitter without the deposited mass changing. Second, the footprint is a disc of radius about 2 mm with a smooth radial taper, not the 0.6 mm wire radius: physically because a droplet spreads on impact over the pool rather than landing as a column, and numerically because 2 mm spans ~25 cells so the participating count changes smoothly.

THE VALIDATION TARGET, AVAILABLE BEFORE ANYTHING IS RUN

Feed rate and travel speed together fix the bead cross-section with no simulation at all. 84.8 mm^3 deposited over 0.02 m/s x 0.5 s = 10 mm of travel gives **8.5 mm^2**.

Zhao measures the single-pass single-layer deposit at about 8 mm wide and 1.80 mm high, which for a parabolic cap is about 9.6 mm^2. Ours coming out slightly smaller is correct, because some of that volume goes into remelted substrate rather than standing proud.

If the simulated bead cross-section is not near 8.5 mm^2, the wire inlet is wrong — and that is known in advance of running it.

The source will live beside `updateArcSource.H` and read the torch position from the arc's own variables rather than recomputing it. If the wire and the arc ever disagree about where the torch is, the metal lands off the hot spot and everything downstream is quietly wrong.

16. Open items

- **HPC access application** (S1) — open since Phase 1, and now quantified: the production mesh is ~16x the current cost.
- **Solver commit hash recorded per case** (S2) — not yet automatic. Every result should name the solver that produced it.
- **Mills (2002) via inter-library loan** — the source for the temperature-dependent properties currently taken second-hand.
- **The case matrix** still lists `waamMhdFoam` and `additiveFoam` in its solver column, which predates the move to ESI. It also needs new columns for surface tension and the wall thermal condition.
- **The arc power figure** in the project documents was measured on the previous mesh. Re-read it from the current log rather than assuming it carried over.
- `hartmann1d-fine` **is not actually finer** than the base case, so the mesh-refinement leg of Gate A is not yet real.
- **Task A2** — port the hand-built case back to the tokenised template, now including the `decomposeParDict` that was missing and stopped a parallel run this week.